

芯片、系统与产业全景

第一章：从抽象方法到真实系统，再到流程与产业

Digital Circuit Getting Started

2026 年 5 月 22 日

Learning Objectives

- 理解为什么数字系统设计必须依赖抽象、分层与模块化
- 理解数字抽象在抽象什么，以及它成立的工程前提
- 理解集成化为什么成为现代电子系统的主导范式
- 区分 die、package、chip、board、system
- 建立对板级系统、芯片类型、设计流程与产业链分工的整体认识

The Game Plan

- 这门课不是从名词表开始，而是从一张工程地图开始
- 第一章先回答三个问题：
 - ① 为什么复杂系统必须依赖抽象
 - ② 芯片在真实世界里到底是什么对象
 - ③ 一颗芯片如何从想法走到产品
- 后续主线：抽象 → 芯片对象 → 板级系统 → 流程与产业

Why Not Start with Logic Gates?

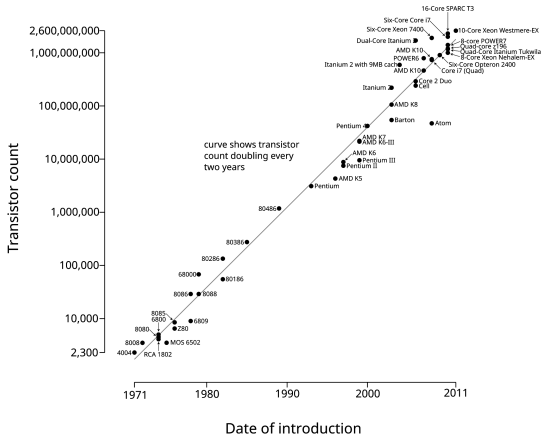
如果一开始就直接进入逻辑门、Verilog 和工具命令，很多初学者会学到后面仍然不知道：

- 芯片到底是什么
- 它为什么不能脱离电源、时钟、存储和接口单独理解
- RTL、验证、综合和流片分别在整条链路的什么位置

第一章的任务：先建立认知坐标，再进入局部技术细节。

Managing Complexity

Microprocessor Transistor Counts 1971-2011 & Moore's Law



一款现代手机 SoC 包含百亿级晶体管。

问题不再只是“功能能不能做出来”，而是“这个功能能否被组织成清楚、稳定、可验证、可修改的
结构”。

Abstraction Hierarchy



System / 系统层



Module / 模块层



Gate / 逻辑门层



Circuit / 电路层



Device / 器件层

每一层只处理本层必须面对的问题，并通过接口把复杂性隔离在边界上。

The Digital Abstraction

- 真实电路处理的是连续电压、电流、延迟与噪声
- 数字设计把它抽象成逻辑 0 / 1、时钟边沿与状态更新
- 这层抽象的价值：
 - 让设计可组合
 - 让验证可分层
 - 让团队能够协作
- 这层抽象的前提：
 - 电平必须可靠
 - 时序必须满足
 - 接口边界必须清楚

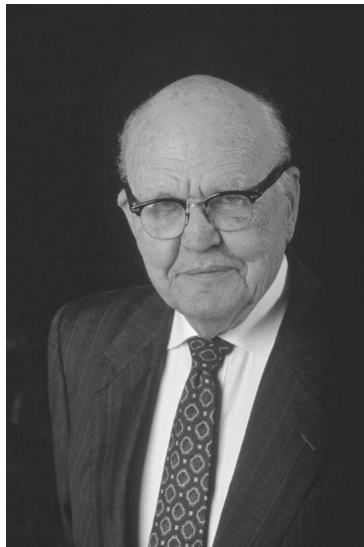
From Physics to Logic

数字抽象不是天然存在的，它是一层人工构建的逻辑秩序：

- 把一部分电压范围定义为逻辑 0，另一部分定义为逻辑 1
- 禁止长期停留在模糊区间
- 让组合逻辑在时钟周期内收敛，再由寄存器捕获

后面会学到的 **setup / hold / metastability / protocol**，都是在保护这层抽象不失效。

Process-Level Integration



Jack Kilby, 1958

- Kilby 与 Noyce 共同开启集成电路时代
- 硅平面工艺让器件与互连更容易稳定复制
- 集成首先是一种**可制造、可复制**的工艺能力
- 后面的单元、IP、系统集成，都建立在这一层之上

Block-Level Integration



Robert Widlar (1937-1991)



μ A709 单片运放

- Widlar 的意义，不只是一个传奇人物
- 而是说明：工艺成熟后，功能单元可以被压进标准芯片
- 从单片运放到今天的 ADC、PMIC、driver 芯片
- 集成化开始从“器件同片”走向“功能成块”

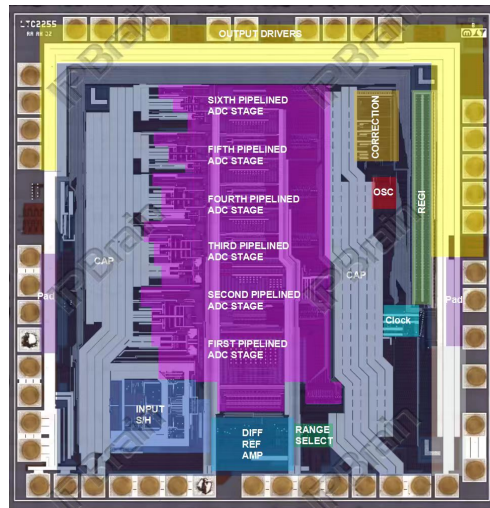
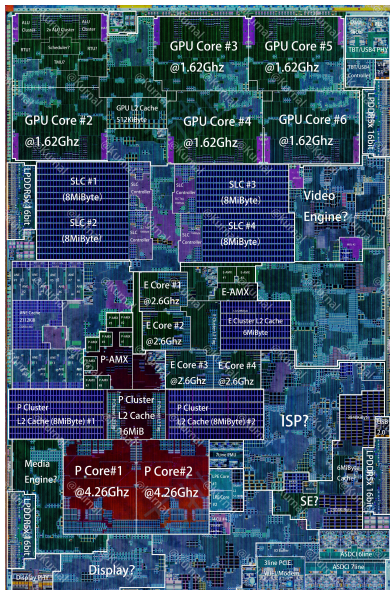
历史在这里是证据：集成化不只堆积晶体管，它还在制造标准功能单元。

IP-Level Integration

- 单元级集成稳定后，设计活动本身也开始集成化
- IP 复用让设计公司不必从零开发每个子系统
- 典型积木：CPU 核、DDR / USB / PCIe PHY、ADC、PLL、存储控制器
- 设计重点开始转向接口、约束、验证流程与工具链组织

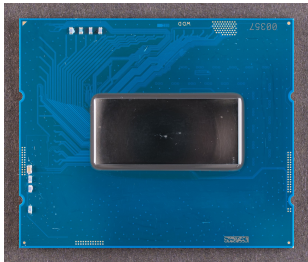
标准接口、EDA 工具和团队分工，不是附属品，而是 IP 级集成成立的条件。

System-Level Integration

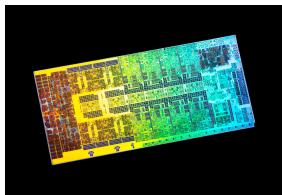


现代模拟 / 混合信号芯片：模拟前端 + 数字校准 + 接口...

From Die to System



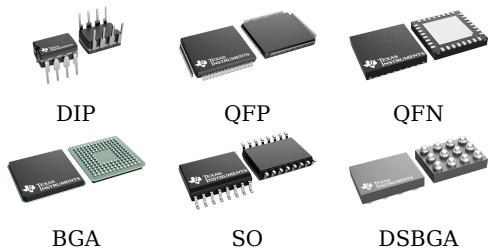
封装后的芯片 (chip)



尚未封装的裸片 (die)

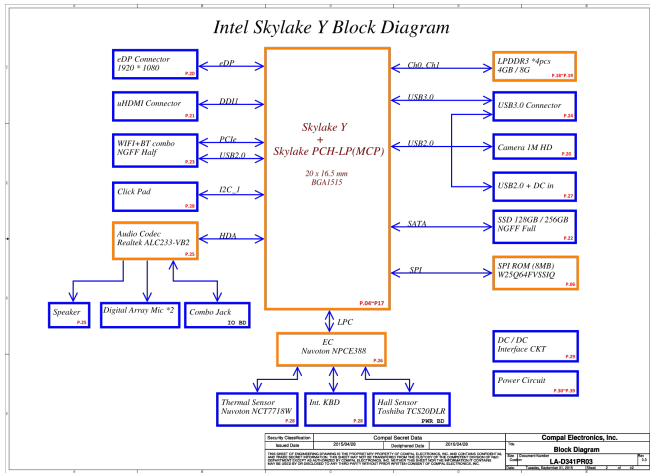
- **die**: 制造完成但尚未封装的裸片
- **package**: 提供保护、电连接与散热路径
- **chip**: 工程语境中常指可上板使用的封装器件
- **board**: 多个器件焊接在 PCB 上形成板级系统
- **system**: 板卡、软件、接口与外设共同工作的整体

Why Packaging Exists



- 保护脆弱的裸片
- 把片上焊盘转换成板级可装配引脚或焊球
- 提供散热路径
- 提供测试与装配的机械形态
- 不同封装是在面积、成本、引脚数、散热和组装之间做权衡

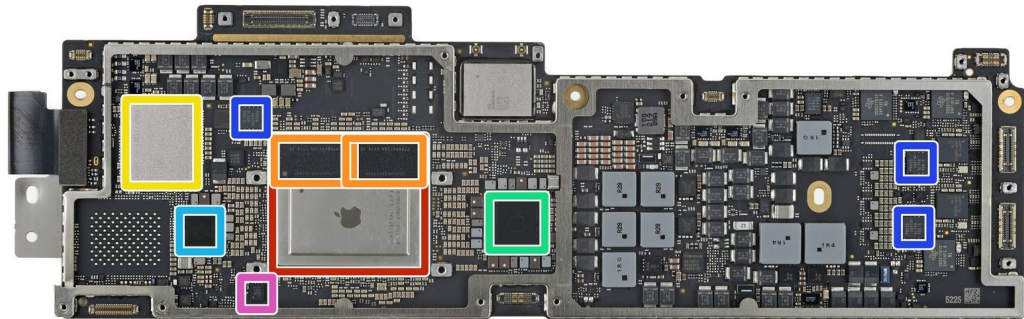
A Chip Never Works Alone



- 主芯片需要供电网络
- 需要时钟和复位
- 需要 DRAM、Flash 等存储器配合
- 还要与 PMIC、接口芯片、传感器等外设协同

芯片设计从来不是孤立看一颗芯片。

From Block Diagram to Real Board



框图是抽象的，真实主板长这样。

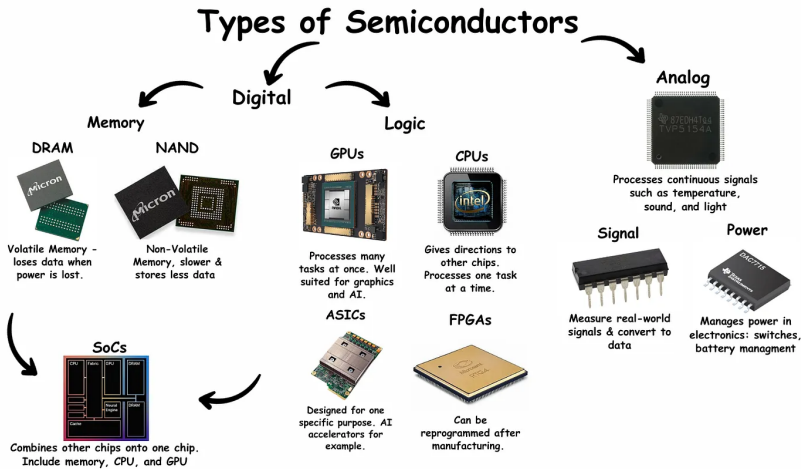
主控、存储、电源管理与接口器件在物理上分布在不同区域，但它们共同构成一个系统。

Minimal Board-Level Checklist

- 主控芯片是什么，它负责计算、控制还是接口聚合？
- 电源从哪里来，是否存在多个电压域？
- 时钟由谁提供，复位如何建立初始状态？
- 片外存储有哪些：DRAM、Flash、EEPROM 还是都没有？
- 它要和哪些外设通信：USB、PCIe、显示、传感器、网络？

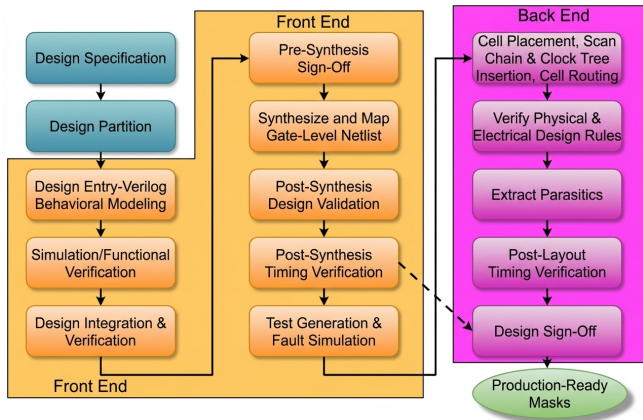
这 5 个问题，是你观察任何板级系统时最先应该建立的分析框架。

Common Chip Types



分类本身不是目的，关键是理解每类芯片在系统中承担的角色：计算、控制、存储、连接、模拟与电源管理。

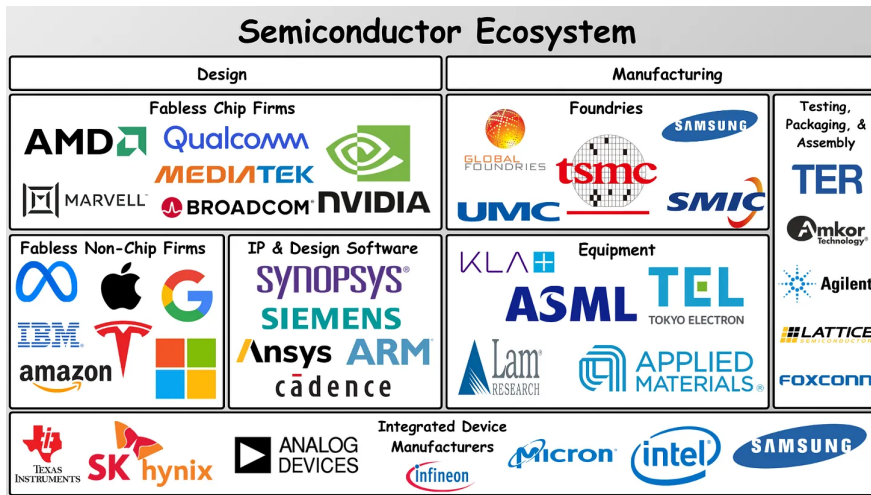
A Simplified Digital Chip Flow



- Spec / Architecture
- RTL Design
- Functional Verification
- Synthesis
- Physical Design
- Signoff / Tapeout

每一步都在消费上一层抽象的结果，并生成下一层可交付物。

Semiconductor Ecosystem



不要把所有公司都称为“芯片公司”。

IP、EDA、Fabless、Foundry、OSAT、设备与材料公司解决的是不同问题。

Roles and Representative Companies

读完这一章后，至少要有最基本的行业常识：

- **IP**: ARM、Imagination
- **EDA**: Synopsys、Cadence、Siemens EDA
- **Fabless**: NVIDIA、AMD、Qualcomm、MediaTek
- **Foundry**: TSMC、Samsung Foundry、SMIC
- **OSAT**: ASE、Amkor、长电科技
- **设备与材料**: ASML、Applied Materials、Lam Research

要求不是背大名单，而是能大致说出一家公司更像在卖 IP、卖工具、做设计、做制造，还是做封测与设备。

产业链主线首先是**角色分工**，但竞争不只发生在硅片制造这一层：

- **TSMC**：不只做代工，也提供 PDK、设计规则和技术支持
- **ARM**：不只卖 IP，也提供工具、参考设计和培训生态
- **设计公司**：往往还提供文档、软件库、参考设计和 FAE 支持
- **STM32 / CUDA** 这类案例说明：很多壁垒发生在芯片之外

这不是说“产业链的本质只有服务竞争”，而是说：在分工结构已经成立的前提下，工具、文档、支持和生态也会显著影响竞争力。

Design Rules

- **抽象优先**: 先确定讨论在哪个抽象层, 再谈具体方案
- **模块化**: 把系统切成可验证、可替换、可复用的模块
- **接口清晰**: 输入输出、时序和约束必须明确
- **尽早验证**: 问题越晚暴露, 修复代价越高
- **用约束管理复杂度**: 面积、性能、功耗、时序是设计的一部分

The 3Y Principle

面对任何模块、芯片或系统，先回答三个问题：

Why

为什么做？目标
和约束是什么？

What

功能边界、输入
输出是什么？

How

如何以可实现、
可验证的方式落地？

这三个问题，是整本课程反复使用的分析框架。

Takeaways

- 数字设计的核心能力之一，是用抽象管理复杂度
- 集成化已经成为现代电子系统的主导范式
- 芯片必须放回 **die / package / board / system** 的对象链条里理解
- 设计流程告诉你后续每一章在整条链路中的位置
- 产业链主线是角色分工，生态与支持是重要补充判断
- 面对任何设计对象，始终先问 Why、What、How

After-class prompt: 画出一个最小板级系统框图，并标出主控、存储、电源与接口器件的角色。